The Wind-AE steady-wind solver: algorithm, continuation, boundary/initial conditions, and the `windsoln` (seed) file format

EXHALE includes a 1-D, steady-state, transonic Parker-wind solver: a Fortran port of Wind-AE *(Murray-Clay et al. 2009; Broome et al. 2025), under* `src/modules/wind_ae/`*. It is used to manufacture a warm-start initial condition for the time-dependent EXHALE run (the* `IC mode: windae` *key, and the standalone* `wind_ae_ic.x`*). This note documents what the solver computes, how it solves it (a boundary-value relaxation from the base to the sonic point, followed by an outward initial-value integration), how the boundary and initial conditions are set, and how an arbitrary planet is reached from a shipped seed by parameter continuation. It also records the structure of the Fortran port and the gates it was validated against. Practical usage is in the user manual (§"Wind-AE warm-start initial conditions"); the EXHALE-side BC/IC reference is* `EXHALE_BC_and_IC.tex`*; the code-by-code comparison with EXHALE and the other escape codes is* `code_comparison.tex`*; the full porting log is* `md_backup/wind_ae_fortran_port_plan.md`*.*

Contents

1. What the solver computes **2**

2. Solution method **2**

2.1 Relaxation on the base→sonic interval (the BVP) . . . 2

2.2 Outward integration (the IVP) . . . 3

2.3 The sonic point and its linearization . . . 3

3. Boundary conditions **3**

4. Initial condition (the seed) and continuation **3**

4.1 Self-consistent base BCs . . . 4

5. The Fortran port **5**

6. Using it in EXHALE **7**

7. Scope and limitations **7**

The `windsoln` (seed) file format **8**

8. File layout **8**

9. Header reference **9**

10. The embedded stellar spectrum **9**

11. The solution grid **10**

1. What the solver computes

Wind-AE solves the *steady-state*, spherically-symmetric, single-streamline (substellar) hydrodynamic equations for an XUV-irradiated H/He atmosphere, from a deep base out through the sonic point to a few planetary radii. The dependent variables are the velocity v, density ρ, temperature T, the neutral fraction Y_s of each species (s = H I, He I), and the photoionization column density $N_{\rm col}$ of each species. Unlike EXHALE (a time-dependent finite-volume code relaxed *to* a steady state) Wind-AE solves the stationary equations *directly* as a two-point boundary-value problem (BVP), so there is no time integration and no Courant limit; the cost is that it bakes in the transonic topology (it cannot represent a non-launching or genuinely time-dependent atmosphere).

The governing relations (code units: r in $R_{\rm pl}$, v in $c_{s,0}$, ρ in ρ_0, T in $T_0 = 10^4$ K) are mass continuity, the transonic momentum (Parker) equation, energy balance with photoionization heating and line/recombination/Lyα/bolometric cooling, and photoionization–recombination equilibrium for each species:

$$\frac{d\rho}{dr} = -\rho\left(\frac{2}{r} + \frac{1}{v}\frac{dv}{dr}\right), \tag{1}$$

$$\frac{dv}{dr} = \frac{v}{v^2 - \gamma T/\mu}\left[\frac{2\gamma T}{\mu r} - (\gamma-1)\frac{Q}{v} - \frac{d\phi}{dr}\right], \tag{2}$$

$$\frac{dT}{dr} = (\gamma-1)\left(\frac{Q\mu}{v} + \frac{T}{\rho}\frac{d\rho}{dr}\right) + \frac{T}{\mu}\frac{d\mu}{dr}, \tag{3}$$

$$\frac{dY_s}{dr} = \frac{\alpha_{{\rm rec},s}\, n_e\, n_{{\rm ion},s} - \Gamma_s}{v\, n_{{\rm tot},s}}\frac{R_{\rm pl}}{c_{s,0}}, \qquad \frac{dN_{{\rm col},s}}{dr} = -Y_s\, n_s\, \frac{R_{\rm pl}}{N_{{\rm col},0}}, \tag{4}$$

where Q is the net specific heating, ϕ the (optionally tidal) potential, μ the mean molecular weight, $\gamma = 5/3$, Γ_s the photoionization rate (primary + secondary), and $\alpha_{{\rm rec},s}$ the recombination coefficient. Equation (2) is singular at the sonic point ($v^2 = \gamma T/\mu$); the solver handles it with an L'Hôpital linearization (§2.3).

2. Solution method

2.1 Relaxation on the base→sonic interval (the BVP)

The interval from the base $R_{\rm min}$ to the sonic point r_s is mapped to a fixed parameter $q \in [0,1]$ via $r = R_{\rm min} + q\,z$, where $z \equiv r_s - R_{\rm min}$ is itself an unknown (a relaxed variable, constant in r). On a grid of $M = 1501$ points the $N_e = 4 + 2N_{\rm spec} = 8$ finite-difference residual equations (one per variable: v, z, ρ, T, Y_s, $N_{{\rm col},s}$) are solved by the Numerical Recipes `solvde` Newton relaxation. Each iteration assembles the block-tridiagonal Jacobian `s` by *numerical* differencing of the residuals (`difeq` + `get_eqn_evals`, step $\delta = x/10^7$), factorizes it (`pinvs`/`red`/`bksub`), and updates the solution; it converges quadratically (the shipped seed relaxes to $\|{\rm err}\| \sim 10^{-11}$ in ~ 10 iterations). The variable ordering is permuted by `indexv` so the interior boundary conditions come first, as Numerical Recipes requires.

2.2 Outward integration (the IVP)

Past the sonic point the problem is no longer a BVP: with r_s and the state there in hand, the same ODEs (2) are integrated outward to $R_{\max}$ as an initial-value problem with an adaptive Bulirsch–Stoer integrator (`odeint` + `bsstep`, with `mmid`/`pzextr`); $\sim$1000 output points are stored. The near-sonic singularity is past, so dv/dr is evaluated directly.

2.3 The sonic point and its linearization

A transonic wind must pass smoothly through $v = c_s$, where the numerator and denominator of (2) vanish together. Two boundary conditions are imposed at r_s (the last grid point): the *Bernoulli* (numerator) condition and the Mach-1 (denominator) condition $v = c_s \cdot$ `breezeparam` (`breezeparam`= 1 for a true transonic wind; < 1 gives a breeze). Approaching r_s, dv/dr is computed by an error-function-weighted blend of the analytic expression and an L'Hôpital linearization ($a(dv/dr)^2 + b(dv/dr) + c = 0$, the positive root = wind), so the derivative stays finite and the relaxation does not stall on the $0/0$.

3. Boundary conditions

Base ($r = R_{\min}$, four conditions). Density, temperature, and the neutral fraction of each species are pinned to base values: $\rho(R_{\min}) = \rho_{\rm bc}$, $T(R_{\min}) = T_{\rm bc}$, $Y_s(R_{\min}) = 1$ (neutral base). $R_{\min}$ itself, with $\rho_{\rm bc}$ and $T_{\rm bc}$, follows from a deep, nearly-hydrostatic, bolometrically heated base: the skin/effective temperature from $F_{\rm opt} = L_\star/(4\pi a^2)$, a slant-path $\tau = 1$ surface for ρ, and a microbar base pressure (`base_bcs` in the reference wrapper).

Sonic point ($r = r_s$, two conditions). The Bernoulli and Mach-1 critical-point conditions of §2.3, plus a fixed sonic-point column density $N_{{\rm col},s}$ per species (self-consistently the integral of the neutral density from r_s outward).

4. Initial condition (the seed) and continuation

Newton relaxation needs a nearby starting guess, and the equations admit infinitely many breeze solutions near the transonic one; a generic guess relaxes to a breeze, not the wind. Wind-AE therefore *bootstraps* from a single shipped converged solution and *continues* it to the target planet:

- **Seed.** One converged windsoln (the Murray-Clay fiducial hot Jupiter) is included as `inputdata/windae_seed.csv`. It provides both the relaxation guess and the base BCs.
- **Adaptive ramp (`ramp_var`).** A target parameter is approached in multiplicative steps $p \leftarrow p(1+\delta)$; after each step the wind is re-relaxed from the current solution. On success the step accepts and δ doubles; on failure the guess is restored, the parameter reset, and δ halves. The run aborts a parameter only after many consecutive failures.
- **Ramp order (`ramp_to`).** $F_{\rm XUV} \rightarrow$ gravity ($R_{\rm pl}$, $M_{\rm pl}$) $\rightarrow$ star ($M_\star$, a, $L_\star$), each by `ramp_var`. The converged solution of one step is the guess for the next (in the port this is an in-memory hand-off; the reference wrapper round-trips it through `guess.inp`).

During the ramp the base boundary conditions can either be held fixed at the seed values (*static-BC mode*) or re-converged at every step (*self-consistent-BC mode*); the port implements both, selected by an optional `static_bcs` flag on `ramp_var`/`ramp_to` (absent or `.true.` = static-BC mode; `.false.` = self-consistent-BC mode).

4.1 Self-consistent base BCs

The static-BC ramp suffices for hot Jupiters near the seed, but as the ramp moves toward a strongly-bound, highly-irradiated planet the seed's base density, pressure, and sonic-point column density become physically inconsistent with the new gravity and flux, and the relaxation stalls. The self-consistent-BC ramp restores consistency by recomputing the lower boundary as the parameters move. It has three pieces, ported from the reference wrapper.

Closed-form base state (`base_bcs`). Taking $R_{\rm pl}$ as the optical slant-path $\tau = 1$ surface, with a bolometrically heated, near-isothermal layer below the wind, the base temperature and density follow in closed form from the stellar optical flux $F_{\rm opt} = L_\star/(4\pi a^2)$:

$$T_{\rm skin} = \left[\frac{F_{\rm opt}\,(\kappa_{\rm opt}+\kappa_{\rm IR}/4)}{2\,\sigma_{\rm SB}\,\kappa_{\rm IR}}\right]^{1/4}, \qquad T_{\rm eff} = \left[\frac{F_{\rm opt}}{4\,\sigma_{\rm SB}}\right]^{1/4}, \tag{5}$$

$$\rho_{R_{\rm pl}} = \frac{1}{\kappa_{\rm opt}}\sqrt{\frac{\mu G M_{\rm pl}}{8 R_{\rm pl}^3 k_B T_{\rm skin}}}, \tag{6}$$

with default opacities $\kappa_{\rm opt} = 4\times10^{-3}$, $\kappa_{\rm IR} = 10^{-2}\,{\rm cm^2\,g^{-1}}$. The simulation base is set at the vertical IR $\tau = 1$ radius $R_{\rm IR}$ if that lies above $R_{\rm pl}$, otherwise at the radius $R_{\rm mbar}$ where the hydrostatic pressure reaches a microbar reference; ρ and T there are returned in code units. The base mean molecular weight blends the molecular value below the wind with the atomic value inside it,

$$\mu = \frac{m_H}{D}(1-b)+b\,\mu_{\rm mol}, \qquad D = \sum_s \frac{m_H}{m_s} X_s\,(2+Z_s-N_{e,s}-Y_s), \qquad \mu_{\rm mol} = 2.3\,m_H,$$

where b is the bolometric-transition flag (1 on, 0 off), X_s the mass fraction and $Z_s - N_{e,s}$ the electrons already removed from the lowest state of species s (zero for H and He; the reference writes $2-Y_s$, the same for H and He; changed 2026-10-27). This routine is **bit-identical to the reference** on the seed and on HD 209458b/HD 189733b: i.e. on both the $b = 1$ (molecular, $\mu = 2.3\,m_H$) and $b = 0$ (atomic, $\mu = m_H/D$) branches.

Sonic-point column density (`self_consistent_Ncol`, `converge_Ncol_sp`). The column-density BC is made self-consistent by integrating the current solution outward and setting

$$N_{{\rm col},s} = \frac{1}{N_{\rm col,0}}\int_{r_s}^{R_{\rm max}} n_{{\rm neut},s}(r)\,dr,$$

then re-relaxing; the set → solve → recompute loop iterates until the mean relative change is below 8%. The ported goal matches the reference to ∼4% (within that self-consistency tolerance: the small gap is the outward-integration extent, not a physics error).

Applying and ramping (`ramp_base_bcs`). The `base_bcs` target is first attempted as a single jump; if the relaxation fails it is reached by ramping $R_{\rm min}$, $T_{\rm bc}$, and $\rho_{\rm bc}$ individually with adaptive sub-steppers (`ramp_bc`, the boundary-value analogue of `ramp_var`).

Molecular/atomic transition (`converge_mol_atomic`). As the ramp moves to a strongly-bound planet the base sinks *into* the wind. The transition finder (a port of the reference `erf_velocity`) locates the first grid point where photoionization heating exceeds the magnitude of PdV (adiabatic-expansion) cooling: where the wind launches; `heat` = $\sum_s n_{\mathrm{neut},s}\, h_s$ (the rates for each species from `wae_glq_rates_eval`) and `cool` = $(k_B T/\mu_{\mathrm{mol}} m_H)\, v\, d\rho/dr$. When that point reaches the base (within $\sim$10 cells), `turn_off_bolo` ramps `bolo_heat_cool`: $1 \to 0$; because the molecular smoothing erf is scaled by that flag, turning it off also drops μ to its atomic value: i.e. the $b = 1 \to b = 0$ transition the target planet requires.

Orchestration. With `static_bcs=.false.`, `ramp_var` runs the *full* base-BC re-convergence (`ramp_base_bcs` + `converge_mol_atomic` + `converge_Ncol_sp`) *reactively*: only when a step stalls (five consecutive failures); and the cheap molecular/atomic check `converge_mol_atomic` alone runs *proactively* (every ten successful steps: it does no relaxation unless it actually turns the bolometric layer off, so a seed-adjacent planet pays almost nothing). `ramp_to` does a final full pass at the end. This split keeps seed-adjacent planets fast (HD 209458b ramps in $\sim$40 s) while turning the molecular layer off early enough for far-from-seed planets (HD 189733b in $\sim$4 min). The reference wrapper's expensive success-path re-convergence (`converge_bcs`) likewise defaults off.

Status (2026-06-13): the self-consistent-BC ramp converges HD 189733b. On HD 189733b, far from the seed (F_{XUV} rises $\sim$22$\times$, the surface gravity roughly doubles), the base-BC re-convergence fires as the ramp proceeds, and on the M_{pl} ramp the transition reaches the base so `converge_mol_atomic` turns the bolometric layer off (`bolo_heat_cool` : $1 \to 0$, matching HD 189733b's atomic solution). The relaxation then tightens to $\|\mathrm{err}\| \sim 9 \times 10^{-11}$ and the full six-parameter ramp completes with a physical wind ($v > 0$ throughout); the in-process bridge runs the same ramp and writes a valid HD 189733b IC. *Caveat:* EXHALE's subsequent time-integration of HD 189733b is limited by a separate, pre-existing base-breathing instability near $r \sim 1.07\, R_{\mathrm{pl}}$ (unrelated to the Wind-AE IC), so a fully converged EXHALE warm start for that planet still depends on resolving that EXHALE-side issue.

5. The Fortran port

The reference Wind-AE solver core is C ($\sim$6.3k lines incl. Numerical Recipes) driven by a Python wrapper that does the continuation. EXHALE's port reimplements both in Fortran, self-contained under `src/modules/wind_ae/` (the only file that touches EXHALE's globals is the bridge of §6.). Module map:

Module(s)	Role
`wae_config,_types,_grid,_status`	sizes/constants, derived types, the shared q-grid, the graceful-failure flag
`wae_rate_coeffs,_rnew,_rrec,_fe,_ion_pots,_lc_*`	auto-generated atomic-data tables (secondary-ionization, recombination, ionization potentials, line-cooling coefficients)
`wae_spectrum`	GLQ multifrequency spectrum + planet parameters
`wae_glq_rates`	photoionization/heating rates (incl. the column-density rate cache)
`wae_soe`	physics: μ, γ, recombination, Q, all the d/dr's, and the sonic-point linearization
`wae_eqns`	residual equations + the `eval_eqn` dispatcher
`wae_difeq`	numerical-Jacobian assembly of the `solvde` matrix
`wae_nr`	the relaxation stack `solvde`/`pinvs`/`red`/`bksub`
`wae_intode`	outward Bulirsch–Stoer integration
`wae_params,_continuation`	input reading; seed load, `ramp_var`/`ramp_to`, and the self-consistent base-BC handling (`base_bcs`, `ramp_base_bcs`, `self_consistent_Ncol`, `converge_Ncol_sp`)
`wae_ic_writer`	project the solution onto the EXHALE grid, write `*_IC.txt`
`wae_exhale_input`	map an EXHALE `input.inp` to Wind-AE parameters

Validation (against the C/Python reference). Each layer was gated with a trace-driver comparison: the photoionization/heating rates and each species's dY_s/dr are *bit-identical*; the heating/cooling Q and the residual equations match to machine precision except where the libm `erf` differs from gfortran's by $\sim$1 ulp (a benign $\sim 10^{-10}$, washed out at the Newton fixed point); the **full windsoln** (relaxation + outward integration) reproduces the C solution to relative $\leq 2\times10^{-11}$ on three planets; and the continuation ramp reproduces the Python wrapper's mass-loss rate to five digits. A subtle quirk of the reference (the rate cache keys on column density only and ignores Y_s changes) had to be reproduced exactly for the numerical Jacobian to match. Since 2026-10-27 the port no longer reproduces it: the stored rates are reused only when the columns, Y_s, $F_{\rm tot}$ and the composition all match, and the recombination cooling takes T in K and an electron density independent of the Lyα switch (`md/windae_defect_fixes_20261027.md`). These comparisons are therefore a record of the port as it stood, not a property of the present code.

The bit-exact gates need one line restored (2026-08-19). Those comparisons were run with Wind-AE's own `defs.h` constants. The port now uses the CODATA values instead, so that the IC it hands over is built on the same constants as the rest of EXHALE ($G = 6.67430\times10^{-8}$, $k_B = 1.380649\times10^{-16}$, $m_{\rm H} = 1.67353284\times10^{-24}$; Update_EXHALE §63.1). The differences are 2.6×10^{-4}, 6.5×10^{-6} and 1.4×10^{-4} relative, so agreement with the reference is now at that level rather than bit-exact. `wae_params.f90` carries the original three values in a comment; restore them to reproduce the gates above verbatim.

6. Using it in EXHALE

In-process: `IC mode: windae`. With this key in `input.inp` (and `Load IC? False`), `init.f90` calls `wae_generate_ic` (module `wae_exhale_bridge`), which maps the planet's EXHALE parameters to Wind-AE ($F_{\rm XUV} = J_{\rm XUV}$, $L_\star$ from `Stellar Teff/radius`, $R_{\rm pl} = R_0$, the He/H ratio, the base density n_0 and temperature T_0, and `Domain mode`→tidal on/off), ramps the seed to the planet, solves the wind, and writes the IC onto the EXHALE grid (`output/*_IC.txt`); `load_IC` then ingests it. The whole input → solve → IC → run is one `EXHALE.x` invocation. Because EXHALE re-solves the ionization (and metals) from the first step, the Wind-AE IC is only a *warm start*: finish with `Solver: Newton` for a quantitative $\dot{M}$.

Standalone: `wind_ae_ic.x`. `make wind_ae_ic` builds a separate executable that does the same out of process, reading an `input.inp` and writing the IC files on a grid taken from an `IC_dump.txt`; run EXHALE on the result with `Load IC? True`. The standalone tool shares the solver modules but not the EXHALE bridge, so it stays self-contained.

7. Scope and limitations

- **H/He only.** The port fixes $N_{\rm spec} = 2$ (H I, He I): one ionization stage per species. It does *not* model the trace metals, Fe/Ca cooling, the He triplet, or the transit spectrum; those remain EXHALE's job. For an IC this is sufficient: EXHALE adds the metals in equilibrium from the first step, and the IC writer rescales the density to EXHALE's exact He/H.
- **Continuation BCs.** By default (`static_bcs=.false.`) the in-process ramp uses the self-consistent-BC ramp (§4.1): `base_bcs` (bit-exact), `converge_Ncol_sp` (within tolerance), `converge_mol_atomic` (bolometric-layer turn-off), and the reactive/proactive re-convergence in `ramp_var`/`ramp_to`. This converges seed-adjacent hot Jupiters *and* strongly-bound, far-from-seed planets such as HD 189733b (which stalled under the older static-BC ramp). The static-BC path remains available as `static_bcs=.true.`. Note this concerns the *Wind-AE* ramp only; whether EXHALE then time-integrates a given planet cleanly is a separate matter (e.g. HD 189733b hits an EXHALE-side base-breathing instability regardless of the IC source).
- **Substellar geometry, Coriolis cutoff.** Wind-AE is a single substellar streamline integrated only to a few $R_{\rm pl}$; beyond the Coriolis turning radius 3-D effects dominate. The mass-loss rate carries the Murray-Clay 0.33 geometric factor when compared to a global rate.
- **erf ∼1 ulp.** The library error function differs from gfortran's at the last bit; irrelevant for an IC, noted for the record.

The `windsoln` (seed) file format

> *A converged Wind-AE wind solution is stored as a single CSV-like file: a block of comment-prefixed header lines that carry every parameter, boundary condition, and the stellar spectrum, followed by the solution profile itself as a table of normalized (code-unit) columns. The same format is used by (i) the original Wind-AE Python wrapper (Murray-Clay et al. 2009; Broome et al. 2025), (ii) the included solution grid under* `inputdata/windae_grid/`*, and (iii) EXHALE's Fortran port, whose* `load_seed` *reads such a file as the starting point of a continuation and whose* `dump_seed` *writes one back. This note is the authoritative field-by-field reference, taken from the Python reader (*`wrapper/wrapper_utils/windsoln.py`*).*

8. File layout

A windsoln file has three kinds of lines, in order:

1. **Header lines** `#key: v1,v2,...`, one per key, carrying scalars, parameters, BCs, flags, and spectrum metadata (Table 1).
2. **Spectrum rows** `## e,wPhi,`σ_1`,`σ_2: `npts` of them, holding the stellar spectrum sampled at the Gauss–Legendre quadrature points (§10.).
3. **Data rows**: the solution profile, one grid point per line, columns named by `#vars` and normalized by `#scales` (§11.).

Any other line beginning with # is a free comment and is skipped. A minimal excerpt:

```
#nspecies: 2
#vars: r,rho,v,T,Ys_HI,Ys_HeI,Ncol_HI,Ncol_HeI,q,z
#scales: 8.34e9,1.0e-15,9.08e5,1.0e4,1,1,1.0e17,1.0e17,1,8.34e9
#plnt_prms: 2.348e30,8.34e9,1.870e33,4.937e11,2.42e4,3.828e33
#phys_prms: 0.800,0.200,HI,HeI,1.673e-24,6.646e-24,2.30
#bcs: 1.030,7.755,2.466e4,0.198,1.0,1.0,1.08e-3,2.05e-4,1.85e-5,1.50e-5
#tech: 1.0,1.0e2,2.5,0.99
#flags: 1,1.00000,0.00000,1
#add_prms: 0
#spec_prms: 59,2,2009-01-01,scaled-solar,full,5.99e-8,9.12e-6, ... ,2.18e-11,3.94e-11
## 2.150e-9,1.771e6,4.258e-24,1.267e-22
## ... (59 rows) ...
1.030,2.466e4,2.232e-6,0.198,1.0,1.0,7.34e3,4.70e2,0.0,2.76
... (one row per grid point) ...
```

Table 1: Header lines of a windsoln file. Field order is exact; "$\times N$" marks a block repeated for each species.

Key	Fields	Meaning (and units)
`#nspecies`	N	number of species ($N = 2$: H I, He I).
`#vars`	names	data-grid column labels (Table 2).
`#scales`	$N_{\rm col}$	cgs value of one code unit for each data column.
`#plnt_prms`	6	$M_{\rm p}$[g], $R_{\rm pl}$[cm], $M_\star$[g], a[cm], $F_{\rm tot}$ [erg cm^{-2}s^{-1}], $L_\star$[erg s^{-1}]. *Field 5 is the total ionizing flux, not $T_{\rm eq}$.*
`#phys_prms`	$3N$+1	mass fractions X_s ($\times N$); species names ($\times N$); atomic masses [g] ($\times N$); then `molec_adjust` (mean-molecular-weight boost of the molecular base layer, ≈ 2.3).
`#bcs`	$2N$+6	$R_{\rm min}, R_{\rm max}$ [$R_{\rm pl}$]; $\rho_{\rm min}$ [ρ_0]; $T_{\rm min}$ [T_0]; base neutral fractions $Y_{s,\rm min}$ ($\times N$); sonic-point columns $N_{{\rm col},s}$ [$N_{\rm col,0}$] ($\times N$); `erf_drop` (2: the erf-velocity molecular→atomic transition).
`#tech`	4	`breezeparam`, `rapidity`, `erfn`, `mach_limit` (relaxation/solver controls).
`#flags`	4	`lyacool` (0/1); **`tidalforce`** (0 = spherical, 1 = tidal/Roche); **`bolo_heat_cool`** (0 = atomic base, 1 = molecular bolometric layer); `integrate_outward` (0/1).
`#add_prms`	1+2n	n, then n (name, value) pairs of optional extra parameters ($n = 0$: none).
`#spec_prms`	meta	`npts`, N, date, source file (`scaled-solar`), kind (`full`), window[2], resolved[2], normalized[2] (wavelength bounds [cm]), ionization potentials χ_s [erg] ($\times N$). See §10..

9. Header reference

Table 1 lists every header key. N is the number of species (`#nspecies`; $N = 2$ for the included H/He data). Planet parameters are in cgs; boundary conditions and the data grid are in the code units of Table 2.

10. The embedded stellar spectrum

The stellar XUV spectrum that the solution was computed with is stored in the file itself, in two parts:

- `#spec_prms`: metadata: the number of quadrature points `npts`, the source label (here `scaled-solar`, a solar-like SED), the spectral kind (`full`, i.e. resolved rather than monochromatic), the integration window in wavelength ($\approx [6, 912]$ Å, i.e. X-ray to the H Lyman limit), and the the ionization potentials χ_s for each species (e.g. 2.18×10^{-11} erg = 13.6 eV for H, 3.94×10^{-11} erg = 24.6 eV for He).
- the ## rows: the spectrum proper, one row per quadrature point: photon energy hc/λ_i [erg]; the *normalized* weight $w_i \Phi_{\lambda_i} / F_{\rm tot}$; and the photoionization cross sections $\sigma_{\lambda_i,s}$ [cm^2] for each species.

Table 2: Data-grid columns (`#vars`) and their code→cgs scales (`#scales`). ρ_0=10^{-15}, $c_{s,0}$=$\sqrt{k_B T_0/m_{\rm H}}$=9.08×10^5, T_0=10^4, $N_{\rm col,0}$=10^{17}.

Column	Symbol	Scale	Meaning
`r`	r	$R_{\rm pl}$	radius
`rho`	ρ	ρ_0	mass density [g cm^{-3}]
`v`	v	$c_{s,0}$	radial velocity [cm s^{-1}]
`T`	T	T_0	temperature [K]
`Ys_HI`	$Y_{\rm HI}$	1	H neutral fraction
`Ys_HeI`	$Y_{\rm HeI}$	1	He neutral fraction
`Ncol_HI`	$N_{\rm HI}$	$N_{\rm col,0}$	H column to the star [cm^{-2}]
`Ncol_HeI`	$N_{\rm HeI}$	$N_{\rm col,0}$	He column to the star [cm^{-2}]
`q`	q	1	relaxation coordinate $\in[0,1]$
`z`	z	$R_{\rm pl}$	base→sonic span ($r=R_{\rm min}+qz$)

The crucial point is the separation of *shape* and *amplitude*: the ## weights are normalized by $F_{\rm tot}$, so they encode only the spectral *shape* (which photon energies carry what fraction of the ionizing flux), while the absolute level is set by the single scalar $F_{\rm tot}$ in `#plnt_prms`. This is why the included grid can keep one fixed `scaled-solar` shape across all $\sim$1000 solutions and vary only $F_{\rm tot}$ (and $M_{\rm p}, R_{\rm pl}$) from planet to planet. Verified: `#spec_prms` is byte-identical across the entire grid.

Caveat (spectrum dependence). The wind structure depends on the SED shape, not only on $F_{\rm tot}$. The included data assume a `scaled-solar` spectrum; an EXHALE run that instead assumes, say, a power law produces different heating/ionization profiles. A Wind-AE warm-start is therefore only self-consistent with an EXHALE run that uses the *same* spectrum. Conversely, a power-law EXHALE model failing to converge does *not* imply that no steady wind exists for that planet: only that that particular spectrum/numerics combination did not settle.

11. The solution grid

Each data row is one grid point; the columns are named by `#vars` and are stored in dimensionless code units. The physical (cgs) value is recovered as (physical) = (code) × (scale), with the scale taken from `#scales` (Table 2). The independent variable is the relaxation coordinate $q\in[0,1]$ mapping the base to the sonic point; the physical radius is $r=R_{\rm min}+qz$, where z (constant along the solution) is the base→sonic span in $R_{\rm pl}$.

12. Use in the EXHALE port

`src/modules/wind_ae/wae_continuation.f90` reads and writes this format:

- `load_seed` parses `#plnt_prms`, `#phys_prms`, `#bcs`, `#tech`, `#flags`, and the first m data rows (into the relaxation arrays). It *ignores* `#scales`, `#add_prms`, `#spec_prms`, and the ## rows: the spectrum is loaded separately from

`inputdata/windae_spectrum.inp`, which is the same `scaled-solar` data in the equivalent "`# KEY: ...`" header form.

- `dump_seed` is the exact inverse: it reconstructs the full header (including `#scales`, `#spec_prms`, and the `##` block from the in-memory spectrum arrays) and writes the m relaxation rows, so a converged solution can be banked as a reusable seed.
- **Seed library and picker.** `inputdata/windae_grid/` holds the Broome et al. (2025) grid ($\sim$1000 solutions spanning $M_{\rm p} \in [3, 528]\,M_\oplus$ and $R_{\rm pl} \in [1.9, 22.6]\,R_\oplus$, all with `tidalforce=1` and the fixed `scaled-solar` spectrum), plus the eight `starting_solutions`. With `Wind-AE seed: grid` in `input.inp`, `pick_nearest_seed` reads the grid manifest (one row of `path`, $M_{\rm p}$, $R_{\rm pl}$, $F_{\rm tot}$ per file; rebuilt by `make_windae_manifest.sh`) and selects the solution closest in $(\log M_{\rm p}, \log R_{\rm pl})$, so the continuation starts from a near neighbor rather than the single distant fiducial seed. `Wind-AE seed out:` dumps the converged solution back into this format.